FAKE NEWS CLAIM CLASSIFICATION

Educational Prototype

Mini Project Report

Submitted by	_____
Register Number	_____
Department / Class	_____
College	_____
Submitted to	_____
Academic Year	2026–2027

Note: This is an educational prototype report. It demonstrates text classification and is not a reliable fact-checking system.

Abstract

Fake news and misleading online claims can spread quickly through social media, messaging apps, and websites. This project presents an educational prototype that classifies a text claim into a simple label such as **Potentially Misleading** or **Needs Verification**. The prototype uses basic natural-language processing (NLP) and a machine-learning text classifier. It demonstrates preparing labelled sample data, converting text into numerical features, training a model, evaluating it, and displaying a prediction. The result is only a screening signal: a model cannot determine truth from wording alone, and real-world verification requires reliable sources, context, and human review.

1. Introduction

People encounter large amounts of information online every day. Some posts contain inaccurate statements, manipulated context, satire, or unsupported claims. Manually checking every claim takes time. A text-classification tool can help flag claims for further review, but it should not be treated as a final judge of truth.

The proposed project is called **Fake News Claim Classification (Educational Prototype)**. It accepts a short claim as input and returns a preliminary label based on patterns learned from a labelled training dataset.

2. Problem Statement

To build a simple, explainable prototype that accepts a text claim and predicts whether it resembles examples labelled as potentially misleading or requires verification, while clearly communicating that the prediction is not proof that the claim is true or false.

3. Objectives

- Build a basic text-classification pipeline for short claims.
- Prepare and clean a small labelled dataset for demonstration.
- Convert text into numeric features using TF-IDF.
- Train and test a baseline classifier such as Logistic Regression.
- Display a predicted category and a clear caution message.
- Evaluate with precision, recall, F1-score, and a confusion matrix.
- Discuss limitations, ethical concerns, and future improvements.

4. Scope and Limitations

The prototype works with text only and uses patterns in its training data. It does not automatically check live websites, verify quotations, identify manipulated images, or understand every language and local context. Performance may be poor on new topics, slang, satire, or claims unlike the training examples. The interface must say that labels are preliminary.

5. Proposed System

- **Input:** A user enters a short news claim or social-media statement.
- **Pre-processing:** Normalize text, remove unnecessary whitespace, and prepare text for vectorization.
- **Feature extraction:** TF-IDF represents words and phrases as numeric features.

- **Classification:** A Logistic Regression model predicts a label learned from labelled examples.
- **Output:** Show the predicted category with a reminder to verify using trustworthy sources.

6. System Requirements

Hardware

A laptop or desktop computer with at least 4 GB RAM is sufficient for a small classroom prototype.

Software

Python 3; Jupyter Notebook or VS Code; pandas; scikit-learn; and optionally Streamlit for a simple web interface.

7. Dataset Design

Use a properly licensed, documented dataset with labels and a clear definition of each class. A tiny synthetic dataset can illustrate the workflow, but must not be presented as representative of real-world news. Avoid collecting private messages or personal data. Split data into training and testing sets before model training.

Example claim (illustrative only)	Suggested demo label
“A local notice says the library closes at 6 PM.”	Needs Verification
“Share this immediately: a secret cure works for every disease!”	Potentially Misleading
“The city website lists a road closure for Tuesday.”	Needs Verification
“Forward this shocking claim; no source is needed.”	Potentially Misleading

These examples only demonstrate how labels might be recorded. The model should be trained and evaluated on a larger, balanced, representative dataset before meaningful performance claims are made.

8. Methodology

- Collect and inspect a labelled dataset; document its source and limitations.
- Remove duplicate records and handle missing text or labels.
- Split the dataset into training and test subsets using a fixed random seed; avoid data leakage.
- Create a pipeline using TF-IDF features and Logistic Regression.
- Train the pipeline on the training subset.
- Evaluate on the held-out test subset using precision, recall, F1-score, and a confusion matrix.
- Test examples outside the training set and record failure cases.
- Present predictions with a disclaimer and encourage source-based verification.

9. Algorithm

TF-IDF: gives higher weight to terms that are informative within a document relative to the dataset. **Logistic Regression:** learns weights for text features and estimates class probabilities. It is a useful baseline, but learns correlations in the dataset rather than independently establishing factual truth.

10. Example Python Implementation

The starter code expects a CSV file named **claims.csv** with two columns: **text** and **label**. Labels should be consistently defined, for example “Potentially Misleading” and “Needs Verification”.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

# claims.csv must contain columns: text, label
df = pd.read_csv("claims.csv").dropna(subset=["text", "label"])
df["text"] = df["text"].astype(str).str.strip()
df = df[df["text"] != ""]

X_train, X_test, y_train, y_test = train_test_split(
df["text"], df["label"], test_size=0.25,
random_state=42, stratify=df["label"]
)
model = Pipeline([
("tfidf", TfidfVectorizer(ngram_range=(1, 2), max_features=10000)),
("classifier", LogisticRegression(max_iter=1000))
])
model.fit(X_train, y_train)
predictions = model.predict(X_test)
print(classification_report(y_test, predictions, zero_division=0))
print("Confusion matrix:")
print(confusion_matrix(y_test, predictions))

def classify_claim(claim):
label = model.predict([claim])[0]
return {"prediction": label,
"warning": "Preliminary text-based result. Verify using reliable sources."}

claim = input("Enter a claim to review: ")
print(classify_claim(claim))
```

Important: This code requires a real, labelled claims.csv file. It is a starter implementation, not a ready-to-use detector. If the dataset is too small or lacks both classes, stratified splitting or model training may fail.

11. Evaluation

Report measured results from the actual test set; do not invent accuracy figures. Accuracy alone can hide poor performance when classes are imbalanced. Precision measures how many items predicted in a class are correct; recall measures how many true items in that class were found; F1-score balances precision and recall. Review the confusion matrix and examine false positives and false negatives.

12. Expected Output

When a claim is entered, the prototype should show a label such as “Potentially Misleading” or “Needs Verification,” followed by: “This is an automated preliminary classification, not a fact-check. Check the original source, date, evidence, and multiple reputable sources.” The system should not display “true” or “fake” as a definitive verdict.

13. Ethical Considerations

- Do not use the model to censor users or make high-impact decisions.
- Avoid storing user-submitted claims unless necessary and consented to.
- Document dataset sources, label definitions, class balance, and known biases.
- Show uncertainty and allow users to challenge or correct labels.
- Do not treat writing style, political viewpoint, or sensational wording alone as proof of falsehood.

14. Future Enhancements

- Add source retrieval and links to credible fact-checking organizations.
- Support multilingual claims with appropriate datasets and evaluation.
- Provide explanations such as influential phrases, without overstating them.
- Build a Streamlit interface with an input box, prediction, and safety disclaimer.
- Test robustness against paraphrasing, satire, and changing topics.

15. Conclusion

This project demonstrates the main stages of a machine-learning text-classification application: dataset preparation, TF-IDF feature extraction, model training, evaluation, and user-facing predictions. Its educational value is showing how NLP can support claim triage while highlighting the limits of automated classification. Reliable misinformation assessment requires evidence, source credibility, context, and human judgment.

References and Learning Resources

- scikit-learn documentation: text feature extraction and model evaluation — https://scikit-learn.org/stable/
- Pandas documentation — https://pandas.pydata.org/docs/
- UNESCO, Journalism, 'Fake News' & Disinformation: Handbook for Journalism Education and Training (2018).
- For any dataset used, cite its original publisher, licence, collection method, and label definitions.

End of report. Replace the cover-page blanks with your details and add your instructor’s required format before submission.